# Լաբորատոր Աշխատանք 6

## 6. Դասընթացների գրանցում և նախապայմաններ

### Աշխատանքի նպատակը

Սովորել օգտագործել struct կառուցվածքային տիպի զանգվածներ և դրանց մշակման համար ստեղծել առանձին ֆունկցիաներ: Աշխատել string տիպի դաշտերի հետ, մուտքագրել, արտածել և մշակել կառուցվածքների զանգվածի տվյալները: Ամրապնդել դասավորման, որոնման և հաշվարկման հմտությունները:

### Աշխատանքի պահանջը

Ստեղծել Product կառուցվածք՝ name, category, manufacturer, price և quantity դաշտերով: Մուտքագրել 12 ապրանքի տվյալներ և զանգվածի մշակման համար ստեղծել առանձին ֆունկցիաներ:

### Առաջադրանք

Ֆունկցիաների միջոցով՝ հաշվել յուրաքանչյուր ապրանքի ընդհանուր արժեքը (գին × քանակ), գտնել ամենամեծ ընդհանուր արժեք ունեցող ապրանքը, հաշվել ամբողջ պահեստի ապրանքների ընդհանուր արժեքը, արտածել տրված քանակի շեմից փոքր քանակ ունեցող ապրանքները և դասավորել ապրանքները ըստ ընդհանուր արժեքի նվազման կարգով:

```
#include <iostream>
#include <string>
using namespace std;
struct Product {
    string name;
    string category;
    string manufacturer;
    double price;
    int quantity;
};

// 12 ապրանքների մուտքագրում
void inputProducts(Product products[], int n) {
    for (int i = 0; i < n; i++) {
        cout << "\nԱպրանք " << i + 1 << endl;
        cout << "Անուն: ";
```

```
        getline(cin, products[i].name);
        cout << "Կատեգորիա: ";
        getline(cin, products[i].category);
        cout << "Արտադրող: ";
        getline(cin, products[i].manufacturer);
        cout << "Գին: ";
        cin >> products[i].price;
        cout << "Քանակ: ";
        cin >> products[i].quantity;
        cin.ignore(1000, '\n');
    }
}
// Մեկ ապրանքի ընդհանուր արժեքը
double productTotalValue(const Product& product) {
    return product.price * product.quantity;
}
// Բոլոր ապրանքների ընդհանուր արժեքը
double calculateTotalValue(const Product products[], int n) {
    double total = 0;
    for (int i = 0; i < n; i++) {
        total += productTotalValue(products[i]);
    }
    return total;
}

// Ամենամեծ ընդհանուր արժեք ունեցող ապրանքի որոնում
int findMostValuableProduct(const Product products[], int n) {
    int maxIndex = 0;
    for (int i = 1; i < n; i++) {
        if (productTotalValue(products[i]) >
            productTotalValue(products[maxIndex])) {
```

```
            maxIndex = i;
        }
    }
    return maxIndex;
}
// Շեմից փոքր քանակ ունեցող ապրանքների արտածում
void printLowQuantityProducts(
    const Product products[], int n, int limit) {
    bool found = false;
    cout << "\n===== ՇԵՄԻՑ ՓՈՔՐ ՔԱՆԱԿ ՈՒՆԵՑՈՂ ԱՊՐԱՆՔՆԵՐ =====\n";
    for (int i = 0; i < n; i++) {
        if (products[i].quantity < limit) {
            cout << "Անուն: " << products[i].name
                << " | Կատեգորիա: " << products[i].category
                << " | Արտադրող: " << products[i].manufacturer
                << " | Գին: " << products[i].price
                << " | Քանակ: " << products[i].quantity
                << endl;
            found = true;
        }
    }

    if (!found) {
        cout << "Նման ապրանքներ չկան:\n";
    }
}
// Դասավորում ըստ ընդհանուր արժեքի նվազման կարգով
void sortByTotalValue(Product products[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (productTotalValue(products[j]) >
```

```
            productTotalValue(products[i])) {
            Product temp = products[i];
            products[i] = products[j];
            products[j] = temp;
        }
    }
  }
}
// Ապրանքների ցուցադրում
void printProducts(const Product products[], int n) {
    cout << "\n===== ԱՊՐԱՆՔՆԵՐԻ ՑԱՆԿ =====\n";
    for (int i = 0; i < n; i++) {
        cout << i + 1 << ". "
            << products[i].name
            << " | Կատեգորիա: " << products[i].category
            << " | Արտադրող: " << products[i].manufacturer
            << " | Գին: " << products[i].price
            << " | Քանակ: " << products[i].quantity
            << " | Ընդհանուր արժեք: "
            << productTotalValue(products[i])
            << endl;
    }
}
int main() {
    const int N = 12;
    Product products[N];
    inputProducts(products, N);
    printProducts(products, N);
    int maxIndex = findMostValuableProduct(products, N);
    cout << "\n===== ԱՄԵՆԱՄԵԾ ԸՆԴՀԱՆՈՒՐ ԱՐԺԵՔ ՈՒՆԵՑՈՂ ԱՊՐԱՆՔ =====\n";
    cout << "Անուն: " << products[maxIndex].name << endl;
```

```
    cout << "Ընդհանուր արժեք: "
         << productTotalValue(products[maxIndex]) << endl;
    cout << "\nԲոլոր ապրանքների ընդհանուր արժեքը: "
         << calculateTotalValue(products, N) << endl;
    int limit;
    cout << "\nՄուտքագրեք քանակի շեմը: ";
    cin >> limit;
    printLowQuantityProducts(products, N, limit);
    sortByTotalValue(products, N);
    cout << "\n===== ԴԱՍԱՎՈՐՎԱԾ ԱՊՐԱՆՔՆԵՐ =====\n";
    printProducts(products, N);
    return 0;
}
```